

30

Prompt Engineering

Writing instructions that get the best from an LLM

The cheapest, fastest way to better AI results

[AI/ML Engineer Learning Series · Deep Dive Edition](#)

What it is	The skill of writing good prompts for LLMs
Why it matters	Same model, far better answers
No training	Just better instructions — instant results
Core idea	Be clear, give context, show examples
Useful for	Your RAG chatbot, agents, daily LLM use

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What is Prompt Engineering?	Why how you ask matters
2	Be Clear and Specific	The number one rule
3	The Recipe of a Good Prompt	Role, task, context, format
4	Give the Model a Role	Setting the persona
5	Zero-shot vs Few-shot	Showing examples
6	Chain of Thought	Make it reason step by step
7	Controlling the Output	Format, length, style
8	Prompting in Code	System and user messages
9	Practical Tips	Quick wins
10	Common Mistakes	What to avoid
★	Cheatsheet	Quick reference

1. What is Prompt Engineering?

A prompt is whatever you type to an LLM — your question or instruction. Prompt engineering is the skill of writing prompts that get the best possible answers. The same model can give a brilliant answer or a useless one depending entirely on how you ask. Learning to ask well is the cheapest, fastest way to better AI results — no training, no code changes, just clearer instructions.

The clearer the ask, the better the answer

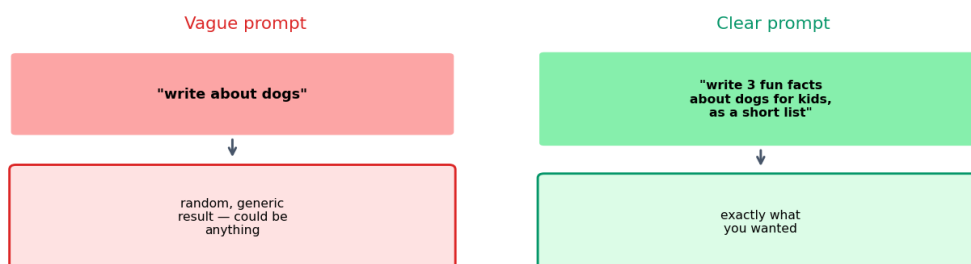


Fig 1 — A vague prompt gives a random result. A clear, specific prompt gives exactly what you wanted.

This skill matters everywhere: chatting with an LLM, building your RAG chatbot's instructions, and guiding AI agents (coming up). A few simple habits make a huge difference. Let's learn them.

2. Be Clear and Specific

This is rule number one, and it solves most problems. The LLM can't read your mind. The more specific you are about what you want, the better the result. Vague in, vague out.

```
# Vague (bad):  
"Tell me about Python"  
  
# Clear and specific (good):  
"Explain Python list comprehensions to a beginner,  
with 2 simple examples and a one-line summary."
```

Notice what the good prompt added: who it's for (beginner), exactly what (list comprehensions), how much (2 examples), and the format (with a summary). Each detail steers the model toward what you actually want. Spend a few extra seconds being specific and you save yourself from useless answers.

3. The Recipe of a Good Prompt

A strong prompt often has a few clear ingredients. You don't always need all of them, but knowing the recipe helps you write better prompts fast.

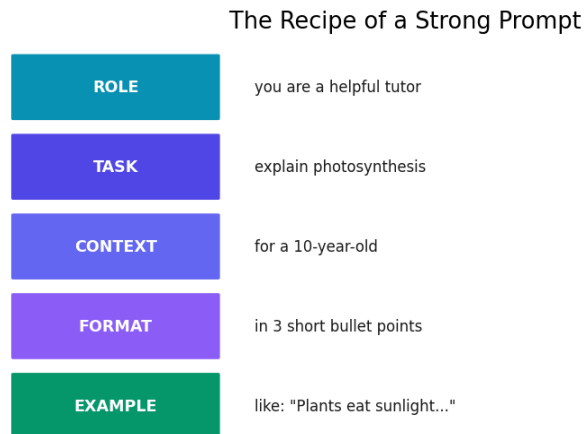


Fig 2 — The ingredients of a strong prompt: role, task, context, format, and optionally an example.

Ingredient	What it does	Example
Role	Sets who the AI should act as	'You are a patient tutor'
Task	The clear main instruction	'Explain photosynthesis'
Context	Background or audience	'for a 10-year-old'
Format	How to shape the answer	'in 3 short bullets'
Example	Shows the style you want	'like: Plants eat sunlight...'

■ You don't need every ingredient every time. For a quick question, just be clear. For an important or repeated task, add role, context, and format to get consistent, high-quality results.

4. Give the Model a Role

One of the simplest tricks: tell the model who to be. Starting with 'You are a...' sets a persona that shapes the whole answer's tone, depth, and vocabulary.

```
"You are an experienced Python instructor.  
Explain decorators to a student who knows basic  
functions but has never seen decorators before."
```

The same question answered as 'a friendly tutor for kids' versus 'a senior engineer reviewing code' gives very different results — each tuned to that role. Pick the role that matches the answer you need. This is exactly how chatbots get their personality: a system prompt tells them who to be.

5. Zero-shot vs Few-shot

Two important styles of prompting. Zero-shot means you just ask, with no examples. Few-shot means you give a few examples first to show the pattern you want.

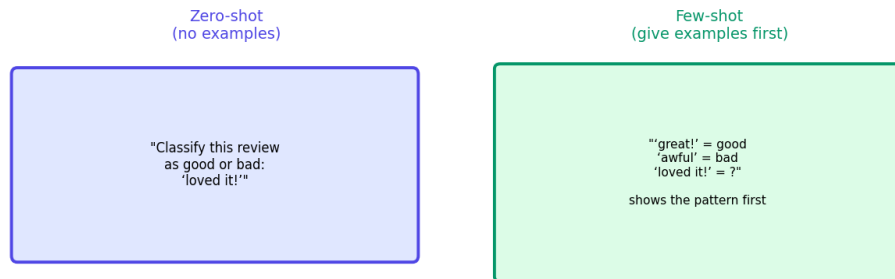


Fig 3 — Zero-shot asks directly. Few-shot shows examples first, so the model copies the pattern.

```
# Few-shot: show the pattern, then ask
"Turn these into formal sentences:

gonna -> going to
wanna -> want to
gimme -> give me
lemme -> ?"
```

Few-shot is powerful when you want a specific format or behaviour that's hard to describe in words. Instead of explaining, you just demonstrate with 2-3 examples and the model copies the pattern. It's like showing someone how to do a task rather than describing it. For simple tasks, zero-shot is fine; for tricky formats, few-shot wins.

6. Chain of Thought

For hard problems — math, logic, multi-step reasoning — there's a magic phrase: ask the model to 'think step by step'. This is called chain-of-thought prompting, and it dramatically improves accuracy on tricky questions.

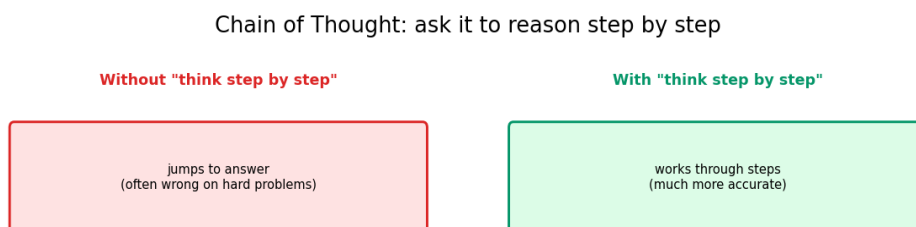


Fig 4 — Asking the model to reason step by step makes it far more accurate on hard problems.

Why does it work? Remember an LLM predicts one word at a time. If you force it to write out its reasoning, each step builds on the last, and it's far less likely to jump to a wrong answer. Without it, the model tries to leap straight to the answer and often stumbles on hard problems.

```
# Add this to hard problems:
```

```
"...Let's think step by step."
```

```
# Or ask explicitly:
```

```
"Show your reasoning before giving the final answer."
```

7. Controlling the Output

You can precisely control the shape of the answer just by asking. Tell the model the format, length, and style you want.

Want this?	Add to your prompt
A list	'Answer as a bulleted list'
Short answer	'In one sentence' or 'in under 50 words'
A table	'Present this as a table'
JSON / data	'Respond only in JSON, no extra text'
Simple language	'Explain like I'm 10 years old'
A certain tone	'Use a friendly, casual tone'

■ Asking for a specific format like JSON is essential when an LLM feeds into code (like an app or agent). 'Respond only in JSON, no other text' makes the output easy for your program to read. This will matter a lot in your AI agent project.

8. Prompting in Code

When you call an LLM through an API, prompts are usually split into a system message (sets the overall behaviour) and a user message (the actual request). This is how your RAG chatbot is structured.

```
messages = [  
  {  
    'role': 'system',  
    'content': 'You are a helpful assistant. Answer ONLY  
               using the provided context. If the answer  
               is not in the context, say you do not know.'  
  },  
  {  
    'role': 'user',  
    'content': f'Context: {retrieved_text}\n\nQuestion: {question}'  
  }  
]
```

■ That system message is classic RAG prompting: it tells the LLM to answer only from the retrieved context and admit when it doesn't know. This is a key trick to reduce hallucination — exactly what your PDF chatbot needs.

9. Practical Tips

- **Iterate** — your first prompt is a draft. Refine it based on the answer you get.
- **Tell it what TO do, not just what NOT to do** — positive instructions work better.
- **Put instructions first**, then the data/text at the end — clearer for the model.
- **Use delimiters** — mark off text with quotes or '---' so the model knows what's what.
- **Ask it to admit uncertainty** — 'say if you're not sure' reduces made-up answers.
- **Break big tasks into steps** — ask for one part at a time for complex work.
- **Give it room to reason** — for hard tasks, let it explain before answering.

■ *Prompt engineering is hands-on and iterative — just like your learning style. The best way to improve is to try a prompt, see the result, and tweak. You get a feel for it quickly with practice.*

10. Common Mistakes

Mistake	Why it's a problem	Fix
Being too vague	Random, generic answers	Be specific about what you want
No format given	Output shape is unpredictable	State the format clearly
Only saying what NOT to do	Confuses the model	Say what TO do instead
Cramming everything in one go	Messy, incomplete answers	Break into steps
Not giving examples	Model guesses your intent	Use few-shot for tricky formats
Giving up after one try	Misses easy improvements	Iterate and refine

Quick Reference Cheatsheet

Technique	What to do
Be specific	say exactly what, for whom, how much
Give a role	'You are a...'
Add context	audience, background, purpose
Set the format	list, table, JSON, length, tone
Few-shot	show 2-3 examples of the pattern
Chain of thought	'think step by step' for hard problems
Positive instructions	say what TO do, not just what not to
Use delimiters	mark text with quotes or ---
Admit uncertainty	'say if you are not sure'

System message	sets overall behaviour in code
Iterate	refine the prompt based on results
Ground with context	'answer only from the text below'

The one idea to remember: the quality of an LLM's answer depends hugely on how you ask. Be clear and specific, give it a role and context, set the format, show examples for tricky tasks, and ask it to reason step by step for hard ones. Better prompts, better results — instantly.

Next Topic: FAISS & Vector Search — how to store millions of embeddings and search them in milliseconds. This is the engine that makes semantic search and RAG fast enough to be practical.